BİTİRME PROJESİ · BÖLÜM 2 · İMPLEMENTASYON

Karpathy modelleri FPGA’de

Dokuz language model mimarisinin, yalnızca integer arithmetic ile ve bilgisayar desteği olmadan tek bir FPGA tasarımında çalıştırılması

Murat Atceken

Nexys A7-100T · Artix-7 XC7A100T · Ekim 2026

ELDE EDİLEN SONUÇLAR

Modeller FPGA'de doğru çalışıyor; kalite kaybı yalnızca INT8'e geçişten gelir ve çok küçüktür

FPGA hesabı bozuyor mu?	**0 fark**	Hayır. 18 modelin 18'i kartın tam simülasyonunda integer referansla bit-exact: 11 241 token'ın hepsi, hash, ekran ve VGA frame'i birebir aynı.
Integer'a geçiş kaliteyi düşürüyor mu?	**≤ %1,6**	Çok az. Loss names modellerinde en fazla %0,3, stories260K'da %1,6 artıyor (1,400 → 1,422 nat). Hiçbir adımda floating point yok.
Modeller chip'e gerçekten sığıyor mu?	**16 / 18**	16 model place and route'tan geçti ve timing'i karşılıyor; en büyüğü LUT'ların %31'ini kullanıyor. İki TinyStories modeli placement'ta takıldı.
Ne kadar hızlı?	**496 tok/s**	Saniyede 496 token (25 MHz). Aynı model Apple M1 CPU'da 14 397 token/s: CPU mutlak hızda önde, ama token başına FPGA 4,4 kat daha az cycle harcıyor.

Kısaca: modeller FPGA'ye özel olarak, doğru ve bilgisayarsız çalışacak şekilde implemente edildi. Kalitenin tek bedeli INT8 quantization'dır; FPGA bu bedele hiçbir şey eklemez.

SUNUMUN PLANI

Sunum, bir modelin PyTorch’tan chip’e uzanan yolunu dokuz adımda izliyor

1 · Hedef ve kısıtlar
Bilgisayarsız çalışan kart tasarımının gereklilikleri

2 · Model ailesi
Dokuz mimari, stories260K ve TinyStories sürümleri

3 · Integer arithmetic
Quantization, requant, softmax ve normalization

4 · Compiler
PyTorch modelinden microcode’a ve memory image’larına

5 · Engine
Bütün modelleri çalıştıran ortak compute unit

6 · Kart sistemi
Generation loop, sampling ve VGA ekranı

7 · Low-bit weight’ler
Larq, LittleBit ve 1-bit matrix-vector unit

8 · Verification
Sekiz aşamalı verification zinciri ve bulunan bug’lar

9 · Place and route ve hız
Fmax, resource utilization ve token/s

Turuncu başlıklı üç bölüm, projenin bu aşamasında eklenen çalışmaları kapsıyor.

1 · HEDEF

Amaç, eğitilmiş bir language model'i bir bilgisayara bağlı olmadan doğrudan FPGA'nin içinde çalıştırmaktır

Yaygın yaklaşım: host-driven

Bilgisayar
generation loop, sampling, metin

UART

FPGA
tek bir hesap adımı

Bilgisayar her token için FPGA'ye komut gönderir ve sonraki token'ı kendisi seçer. FPGA yalnızca bir hesaplayıcı olarak kalır.

Bu çalışma: bağımsız kart

model → engine → sampling → ekran → VGA

Weight'ler bitstream ile BRAM'e yüklenir. Generation loop, sampling, metne çevirme ve ekrana yazma chip'in içinde gerçekleşir.

Yalnızca inference

FPGA eğitilmiş modeli çalıştırır; training ve QAT bilgisayarda yapılır.

Yalnızca Karpathy modelleri

makemore, minGPT, microgpt, llama2.c ve nanochat depoları.

Mac üzerinde verification

Kartın tamamı simülasyonla doğrulanır; fiziksel kart testi kapsam dışıdır.

1 · HEDEF

Hedef kart Nexys A7-100T, CPU'su bulunmayan orta büyüklükte bir Artix-7 FPGA kartıdır



Şekil. Tasarımda kullanılan input ve output'lar; ortada FPGA chip'i.

XC7A100T resource'u	adet	işlevi
LUT (6 input'lu)	63 400	logic
flip-flop	126 800	register
BRAM (36 Kb)	135	~607 KB memory
DSP48E1	240	25 × 18 multiplier
CPU	yok	–

Kartın 128 MB DDR2 memory'si bilinçli olarak kullanılmadı; modelin tamamı chip içindeki BRAM'de tutuluyor.

1 · HEDEF

Tasarımı dört kısıt belirledi: CPU, floating point ve external memory kullanılmıyor; tek bir clock var

CPU bulunmuyor

Generation loop, sampling, detokenizer ve display driver’ı birer hardware state machine olarak tasarlandı.

Floating point kullanılmıyor

DSP48 block’ları integer multiplier’larıdır. Bütün hesap INT8 ve INT16 ile yapılır; exponential, square root ve division da integer’dır.

Model chip içindeki BRAM’e sığmalı

stories260K’nın weight’leri 254 KB, KV cache’i 80 KB tutar; toplam 607 KB’ın altında kalır. Karşılaştırma için GPT-2 124M, 124 MB gerektirirdi.

Tek clock ile synchronous design

MMCM, 100 MHz’ten core clock’unu (50 ya da 25 MHz) üretir. VGA’nın 25 MHz pixel clock’u aynı clock’tan bir enable signal’iyle türetilir.

Sonuç

Modeller küçüktür (50 bin ile 260 bin parametre), ancak sistem eksiksizdir: bir bilgisayarın yapacağı her işi chip kendisi yapar. External memory bulunmadığı için memory delay’i yoktur ve her işlemin cycle sayısı önceden bilinir.

2 · MODEL AİLESİ

Dokuz mimari Karpathy’nin dört deposundan alındı ve aynı ~50 bin parametre bütçesiyle eğitildi

model	kaynak	normalization	activation	pozisyon / durum	attention	param.
bigram	makemore	–	–	yalnız son token	–	729
mlp	makemore	–	tanh	16 token pencere	–	50 459
rnn	makemore	–	tanh	recurrent state	–	49 347
gru	makemore	–	σ, tanh	gated state	–	48 823
bow	makemore	–	tanh	causal ortalama	–	49 979
gpt2	minGPT, nanoGPT	LayerNorm	GELU	learned	MHA	53 120
microgpt	microgpt gist	RMSNorm (param.siz)	ReLU	learned	MHA	51 392
llama	llama2.c	RMSNorm (gain)	SwiGLU	RoPE	GQA	50 304
nanochat	nanochat	RMSNorm (param.siz)	ReLU²	RoPE	GQA, QK-norm	46 784

Tablo. Bütün modeller names.txt (32 bin isim, 27 karakter) üzerinde eğitildi. Gri satırlardaki transformer’lar 4 layer, C = 32 ve 16 token’lık context kullanır.

2 · MODEL AİLESİ

Mimariler aynı yapı taşlarının farklı birleşimleridir; engine bu yapı taşlarının her birini bir op olarak sunar

model	EMB	GEMV	NORM	ATTN, KV	ROPE	MUL	state	BOW	op sayısı
bigram	•								1
mlp	•	•							3
rnn	•	•					•		5
gru	•	•				•	•		8
bow	•	•						•	7
gpt2	•	•	•	•					43
microgpt	•	•	•	•					44
llama	•	•	•	•	•	•			59
nanochat	•	•	•	•	•				60
stories260K	•	•	•	•	•	•			73

Tablo. Her modelin microcode'unda cycle harcayan op türleri (Verilator profili).

stories260K, Karpathy'nin TinyStories üzerinde eğittiği bir Llama 2 modelidir; diğer mimariler de aynı boyutta eğitildi

stories260K (llama2.c, hazır checkpoint)

özellik	değer
embedding boyutu	64
layer	5
head / KV head	8 / 4
vocabulary	512 token
context (kartta)	256
loss: float → INT8	1,400 → 1,422

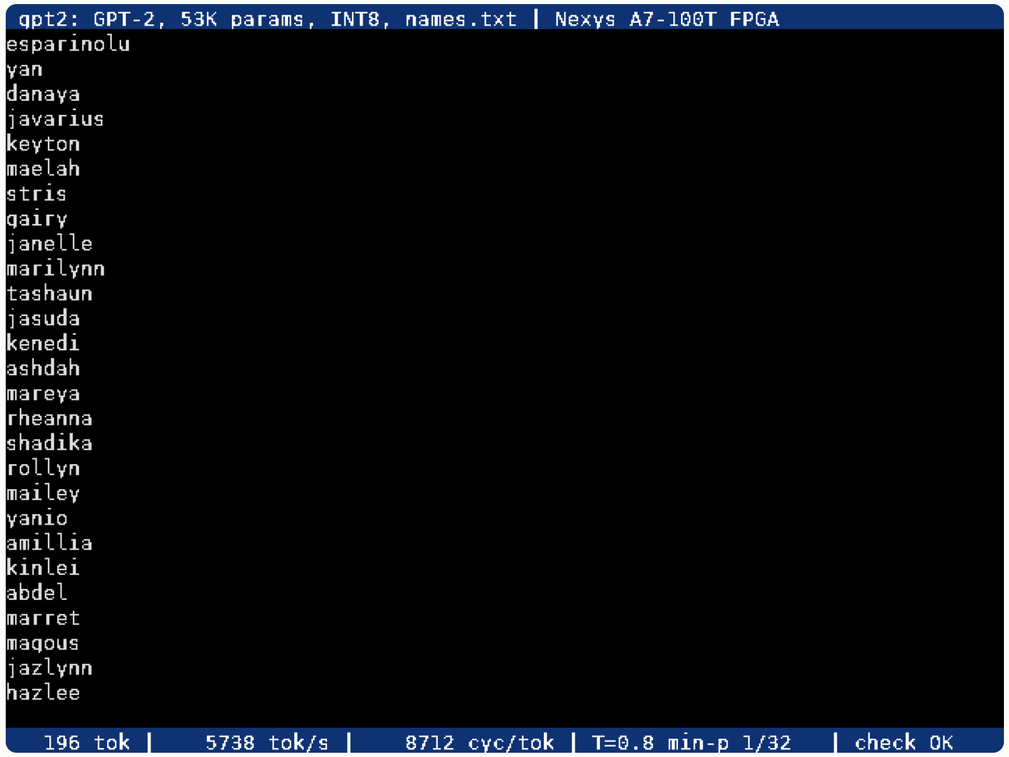
Aynı veri üzerinde eğitilen sürümler (ts_*)

mimari	parametre	test loss
ts_llama	260 032	1,346
ts_gpt2	259 728	1,438
ts_microgpt	261 120	1,575
ts_gru	257 920	1,610
ts_nanochat	260 096	1,637

Not. Sağdaki değerler kendi training ve test düzenimizle ölçülen float test loss'udur. stories260K'nın loss'u 400 validation hikâyesinde ölçüldüğü için iki tablo doğrudan karşılaştırılmamalıdır.

2 · MODEL AİLESİ

Aynı circuit, BRAM’deki modele göre isim ya da hikâye üretir; iki ekran görüntüsü de kartın VGA output’undan alındı



gpt2 (names.txt): 196 token, token başına 8 713 cycle



stories260K (TinyStories): 256 token, token başına 50 397 cycle

Integer arithmetic, çarpma-toplama işlemini tek bir DSP bloğunda tek cycle’da yapar ve modeli dört kat küçültür

float32 ile a · b + c

1. Üsler toplanır, mantisler çarpılır.
2. Toplanacak sayıyla üsler hizalanır.
3. Sonuç toplanır ve normalize edilir.
4. Rounding ve özel durumlar uygulanır.

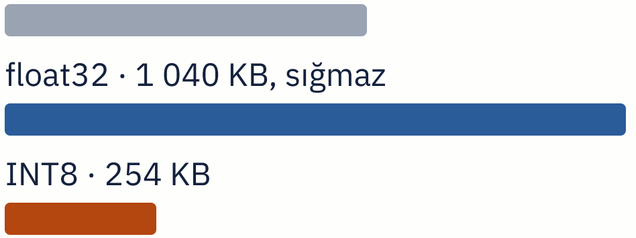
Chip’te hazır bir floating point unit yoktur; bu adımlar LUT’larla kurulur ve pahalıdır.

INT8 ile a · b + c

```
x = [12, −7, 30]
w = [18, −55, 127]
Σ wᵢxᵢ = 216 + 385 + 3 810
= 4 411
```

Her terim, bir DSP48 bloğunda tek cycle’da 8 × 8 bit çarpılıp 32 bitlik accumulator’a eklenir.

stories260K’nın memory’si



Yaklaşımın gerektirdiği

Her sayı artık bir scale ile çarpılmış bir integer olarak temsil edilir. Scale’lerin nasıl seçildiği ve integer’lar arasındaki işlemlerin floating point’e dönülmeden nasıl yapıldığı sonraki beş slaytta anlatılıyor.

Quantization, her gerçel sayıyı bir scale ile çarpılmış bir integer olarak temsil eder

Örnek: mutlak değeri en büyük elemanı 0,268 olan bir weight satırı

`S = 0,268 / 127 = 0,00211`

w	w / S	q	S · q	error
0,0372	17,63	**18**	0,0380	+0,0008
−0,1153	−54,64	**−55**	−0,1161	−0,0008
0,2680	127,00	**127**	0,2680	0
0,0009	0,43	**0**	0	−0,0009

Tablo. Error hiçbir zaman S/2 = 0,00106'yı aşmaz; çok küçük weight'ler sıfıra round edilir.

$$q = \text{clip}(\text{round}(r / S), -127, 127)$$
$$\hat{r} = S \cdot q, \; |r - \hat{r}| \le S/2$$

Weight'ler: output channel başına scale

Her satırın kendi scale'i vardır; büyük değerli bir satır küçük değerli bir satırın resolution'ını bozmaz.

Activation'lar: per-tensor sabit scale

Scale, kalibrasyon verisinde |x|'in %99,99 percentile'ından seçilir. Nadir uç değerler kırpılır ve kalan değerler daha ince bir resolution kazanır.

Requant, real-valued scale'i bir integer multiplier ve bir shift ile uygular; hiçbir adımda floating point kullanılmaz

acc (INT32) → + bias → × m_0 → + 2^{sh-1} → ≫ sh → sat_8 → q_y (INT8)

$$y \approx S_x\, S_w \cdot \mathrm{acc},\ \mathrm{acc} = \Sigma\, q_w\, q_x + q_b$$
$$q_y = \mathrm{round}(M \cdot \mathrm{acc}),\ M = S_x\, S_w \,/\, S_y$$
$$M \approx m0 \cdot 2^{-\mathrm{sh}},\ m0 \in [2^{15}, 2^{16})$$

Bu temsilde relative error 2^{-16}'dan küçüktür. Her output channel'ının m_0 ve sh değerleri parametre ROM'unda saklanır; hardware'de işlem bir çarpma, bir toplama ve bir shift'ten ibarettir.

Sayısal örnek

S_x = 0,05 S_w = 0,00211 S_y = 0,077
M = 0,0013703 → sh = 25, m0 = 45 979
acc·m0 = 5 321 · 45 979 = 244 654 259
+ 2^{24} = 261 431 475
≫ 25 = 7
float: 5 321 · M = 7,29 → 7

Integer softmax, exponential’ı bir shift ve 256 entry’lik bir lookup table’a ayırır ve head başına yalnızca bir division yapar

1 · En büyüğü çıkar
$u_i = (m - s_i) \cdot C \geq 0$

→

2 · Üssü ayır
$e_i = \mathrm{LUT}[f_i] \gg n_i$

→

3 · Biriktir
$E = \Sigma\, e_i$, $A = \Sigma\, e_i\, v_i$

→

4 · Bir kez böl
$y = A \cdot \lfloor 2^{32}/E \rfloor \gg 32$

Yöntemin dayanağı. $e^x = 2^{(x \cdot \log_2 e)}$ eşitliğinden, $u_i = 256\, n_i + f_i$ yazılır. 2’nin tam kuvveti bir shift’tir; fractional kısım $\mathrm{LUT}[f] = 2^{15} \cdot 2^{(-f/256)}$ LUT’undan okunur. En büyük değer çıkarıldığı için her üs sıfır ya da negatiftir ve overflow oluşmaz.

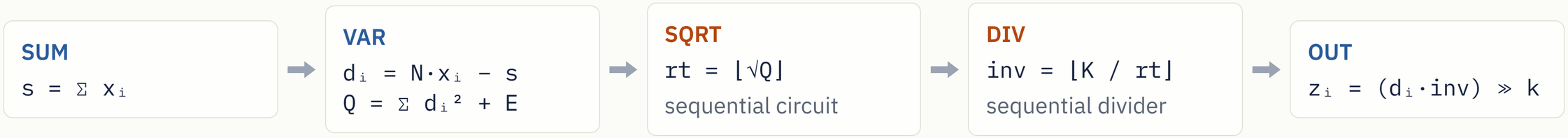
Fractional kısmın 8 bite kesilmesi, %0,27’den küçük bir relative error getirir.

Örnek: üç score, C = 92

s	u	n, f	e	e / E	float
9	0	0, 0	32 768	0,607	0,607
6	276	1, 20	15 520	0,287	0,287
2	644	2, 132	5 730	0,106	0,106

$E = 54\,018$ ve $R = \lfloor 2^{32} / 54\,018 \rfloor = 79\,509$’dur; üç probability float softmax ile üç basamakta örtüşür.

Integer LayerNorm ve RMSNorm, vector başına bir square root ve bir division kullanır; output'un overflow olmayacağı ispatlanmıştır



Lossless integer işlemler

$d_i = N(x_i - \mu)$ division gerektirmez ve bilgi kaybetmez. RMSNorm'da s = 0 alınır; aynı unit iki normalization'ı da hesaplar.

Overflow olmaz (Samuelson)

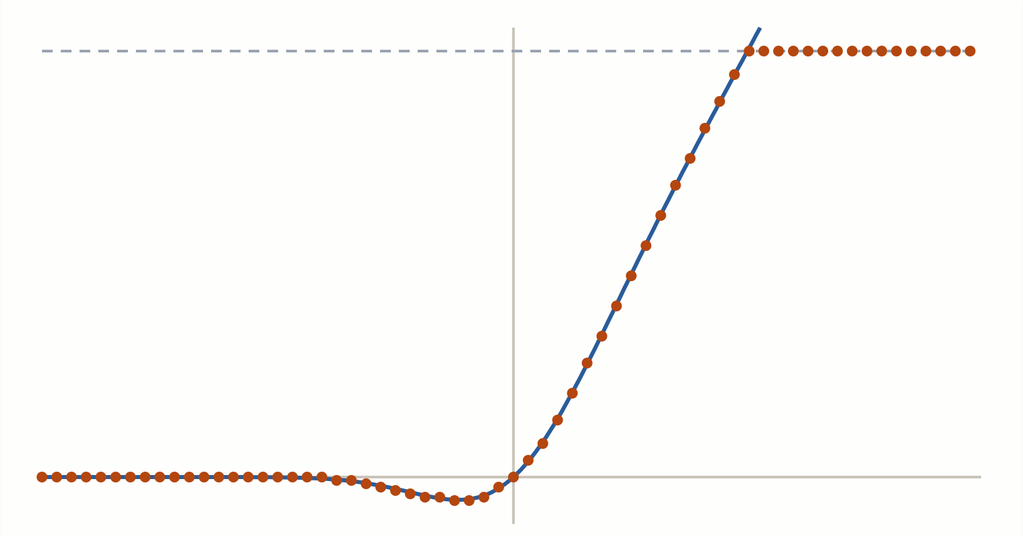
$|x_i - \mu| \le \sigma\sqrt{(N-1)}$ olduğundan output'un büyüklüğü veriden bağımsız olarak sınırlıdır; kalibrasyon gerekmez.

γ ve β fold edilir

$W(\gamma \odot \hat{z} + \beta) + b = (W \cdot \mathrm{diag}\,\gamma)\,\hat{z} + (W\beta + b)$. Hardware yalnızca $\hat{z}$'yi hesaplar.

Maliyet. NORM, gpt2'de cycle'ların %14'ünü, QK-norm nedeniyle nanochat'te %33'ünü kullanır. Square root ve division yavaştır, ancak her vector için yalnızca bir kez çalışır.

INT8 input'lu her activation fonksiyonu, 256 entry'lik bir lookup table ile yaklaşıma gerek kalmadan temsil edilir



Şekil. Mavi çizgi gerçek GELU fonksiyonunu, turuncu noktalar tablonun her dördüncü satırını gösterir (S_h = 0,05, S_g = 0,025). Kesikli çizgi 127'deki saturation'ı belirtir.

q	x = q·S_h	GELU(x)	tablo
−40	−2,0	−0,045	−2
−10	−0,5	−0,154	−6
0	0	0	0
20	1,0	0,841	34
60	3,0	2,996	120

Input yalnızca 256 farklı değer alabildiği için tablo fonksiyonu **tam** olarak temsil eder. Error yalnızca input ve output'un quantization'ından kaynaklanır.

Engine'de LUT GEMV'nin output'una bağlanır: accumulator önce requant ile INT8'e indirilir, ardından LUT'tan geçer. LUT layer başına 256 byte tutar. Aynı yöntem GELU, SiLU, tanh, σ ve ReLU² için kullanılır.

INT8'e geçişin kaliteye etkisi küçüktür: loss names modellerinde en fazla 0,006, stories260K'da 0,022 nat artar

model	float	INT8	Δ loss (nat/token)	Δ	argmax aynı
bigram	2,451	2,451		0,0000	%100,0
mlp	1,986	1,987		+0,0011	%98,1
rnn	1,964	1,967		+0,0032	%96,1
gru	1,956	1,956		+0,0003	%98,3
bow	2,045	2,045		−0,0003	%97,0
gpt2	2,000	2,002		+0,0017	%96,3
microgpt	1,971	1,974		+0,0032	%95,3
llama	2,003	2,004		+0,0013	%93,5
nanochat	1,969	1,975		+0,0055	%93,8
stories260K	1,400	1,422		**+0,0217**	%92,0

Tablo. Float model ile hardware ile bit-exact çalışan integer programın aynı test verisindeki loss'ları. Son sütun, en olası token'ın iki modelde aynı çıktığı adımların oranıdır.

4 · COMPILER

Compiler, eğitilmiş bir PyTorch modelini altı adımda hardware'in okuyacağı microcode'a ve memory image'larına dönüştürür

1 · Model
Eğitilmiş .pt dosyası ya da Karpathy'nin .bin dosyası

→

2 · IR
Op listesi; PyTorch ile farkı 10^{-4}'ün altında

→

3 · Folding
γ, β ve $1/\sqrt{d_h}$ weight'lere aktarılır

4 · Kalibrasyon
Activation scale'leri %99,99 percentile'ından

→

5 · Integer program
IntRunner, hardware'in golden reference'ı

→

6 · Export
.mem dosyaları BRAM'e yüklenir

dosya	içerik	gpt2'de
prog.mem	op word'leri (microcode)	43 op
weights.mem	INT8 weight'ler, 8'li şeritler hâlinde	6 272 × 64 bit
params.mem	per-channel bias, m_0, sh	1 184 satır
tables.mem, luts.mem	embedding'ler, exp ve activation LUT'ları	1 376 + 1 024

4 · COMPILER

Microcode, modelin forward fonksiyonunun op dizisi olarak yazılmış hâlidir; şekil gpt2 ile llama'nın ilk block'unu karşılaştırır

gpt2 · layer 0 (43 op'un 10'u)

```
NORM ln R → A
GEMV q A → Q
GEMV k A → K
GEMV v A → V
KVSTORE layer 0
ATTN 4 head, d=8 → B
GEMV proj B → R += residual
NORM ln R → A
GEMV fc A → F GELU LUT'u
GEMV mproj F → R += residual
```

llama · layer 0 (59 op'un 14'ü)

```
NORM rms R → A
GEMV q, k, v A → Q, K, V
ROPE q: 4 head
ROPE k: 2 head
KVSTORE layer 0
ATTN 4 head / 2 KV head
GEMV wo B → R += residual
NORM rms R → A
GEMV w1 A → F SiLU LUT'u
GEMV w3 A → G
MUL F ⊙ G → F SwiGLU
GEMV w2 F → R += residual
```

Turuncu satırlar iki mimari arasındaki farkları gösterir: RoPE, grouped-query attention ve SwiGLU'nun element-wise multiplication'ı. Diğer op'lar ortaktır ve engine iki modeli aynı circuit ile çalıştırır.

4 · COMPILER

En büyük model olan stories260K, BRAM'in yaklaşık %71'ini kullanır ve bunun en büyük kısmını weight'ler oluşturur

Şekil. stories260K'nın on-chip memory kullanımı (çubuğun tamamı 607 KB)



■ weight'ler 256 KB ■ KV cache 80 KB ■ tablolar (embedding, exp) 68 KB ■ requant parametreleri 28 KB ■ microcode, LUT'lar 3 KB

□ boş ~172 KB

model	BRAM36	kapasitenin
bigram	2	%1
gpt2 (names)	19,5	%14
ts_gru	69,5	%51
stories260K	97,5	%72
stories260K-bin (1 bit)	56	%41

Weight'ler bitstream'in bir parçasıdır: **$readmemh** ile okunan .mem dosyaları BRAM'lerin başlangıç içeriğini oluşturur. KV cache ve vector buffer'ları ise çalışma sırasında yazılan RAM'lerdir. Tablodaki değerler Yosys synthesis'inin tahminidir.

5 · ENGINE

zoo_core, microcode'u sırayla okuyan ve her op'u ilgili compute unit'ine yönlendiren tek bir engine'dir

prog.mem → fetch ve decode: op, buffer'lar, boyutlar, address'ler → op tamamlanınca sıradaki op; END ile token hazır

Memory (BRAM)

weights · 8 × INT8 / word

wbin · 64 × 1 bit / word

params · bias, m_0, sh

tables, luts

KV cache (yazılır)

→

GEMV 8 MAC lane (DSP)	**GEMVB** 64 ± lane (1 bit)	**requant** seri; LUT output'u
NORM LN, RMS, √, ÷	**ATTN, KVSTORE** 8 lane, exp LUT'u	**ROPE** sin/cos LUT'u
EMB, TABLOGIT table lookup	**MUL, REQ, SCALE** element-wise	**COPY, GRUUPD, BOW** state, ortalama

⇄

Vector buffer'ları

11 vector buffer'ı

residual: INT16

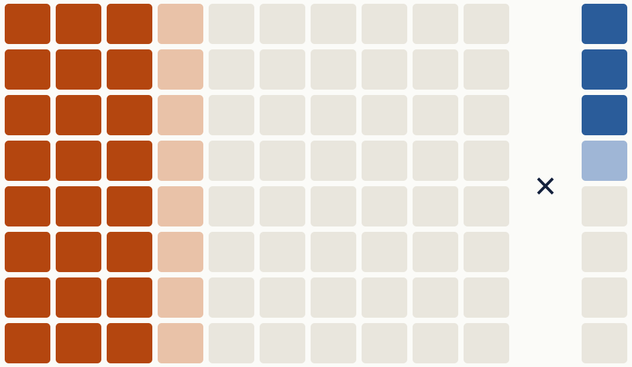
diğerleri: INT8

logit'ler: INT16 → lm_gen

Tanım. Engine, bu projede sıfırdan tasarlanan ve microcode ile programlanan inference accelerator core'udur; kodda **zoo_core** modülüdür (rtl/zoo/zoo_core.sv).

5 · ENGINE

GEMV unit’inde her cycle’da bir input sekiz lane’e broadcast edilir ve sekiz output aynı anda birikir



Şekil. Sekiz satırlık bir output block (turuncu) ve input vector (mavi). Koyu sütunlar işlenmiş input’ları, açık sütun bu cycle’daki input’u gösterir. Input x_i sekiz lane’e broadcast edilir ve 64 bitlik tek bir weight word’ü sekiz weight’i birlikte getirir.

$$\text{cycle} \approx \lceil n_out / 8 \rceil \cdot n_in + \text{sabit}$$

gpt2 fc (32 → 128): 16 block × 32 = 512 cycle.
stories260K w1 (64 → 172): 22 block × 64 = 1 408 cycle.

Bir block tamamlandığında sekiz accumulator bir shadow bank’a kopyalanır ve requant unit’ine sırayla aktarılır; lane’ler bu sırada bir sonraki bloğa başlar. Bu nedenle cycle sayısı önceden hesaplanabilir.

5 · ENGINE

Attention unit’inde her geçmiş token, her head için iki cycle ekler; bu nedenle maliyet context uzunluğuyla doğrusal artar

SCORE

$s_i = q \cdot k_i$, 8 multiplier, cycle başına bir key

t + 1 cycle

EXP

$e_i = \mathrm{LUT}[f] \gg n$, $E \mathrel{+}= e_i$

score hesabıyla örtüşür

DIV

$R = \lfloor 2^{32} / E \rfloor$, head başına bir kez

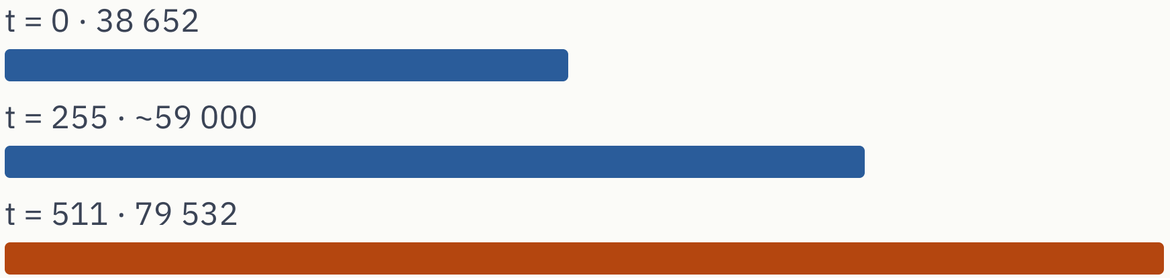
sabit süre

AV

$y = (\Sigma\, e_i\, v_i) \cdot R \gg 32$, 8 lane

t + 1 cycle

Şekil. stories260K’da token başına cycle sayısının pozisyona göre değişimi

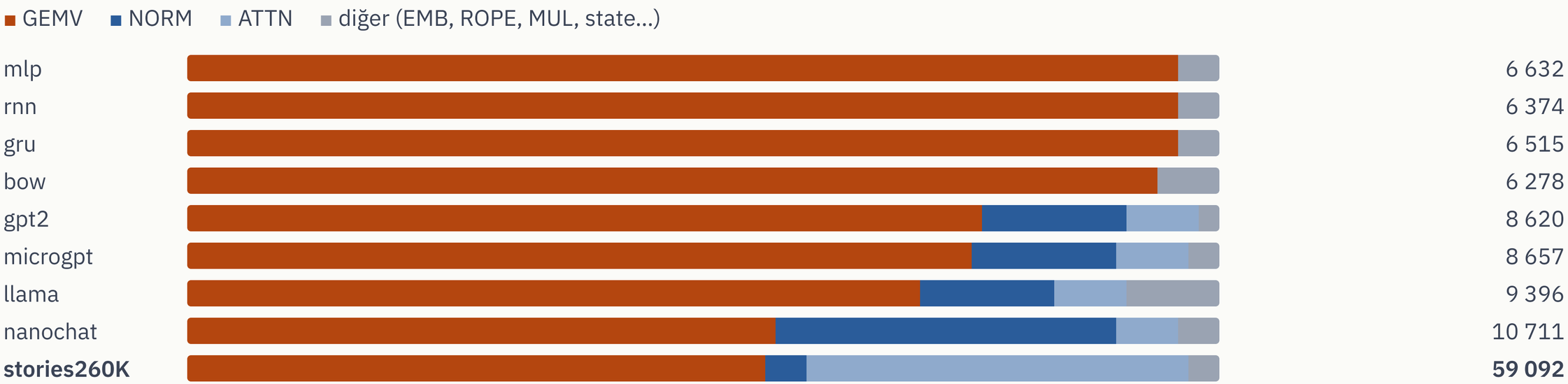


eğim = 5 layer × 8 head × 2 = 80 cycle / token

(79 532 − 38 652) / 511 = 80,0; ölçüm modelle birebir örtüşür. Kartta context’in 256 ile sınırlanması, KV cache’i 80 KB’ta ve en uzun adımı ~59 000 cycle’da tutar.

5 · ENGINE

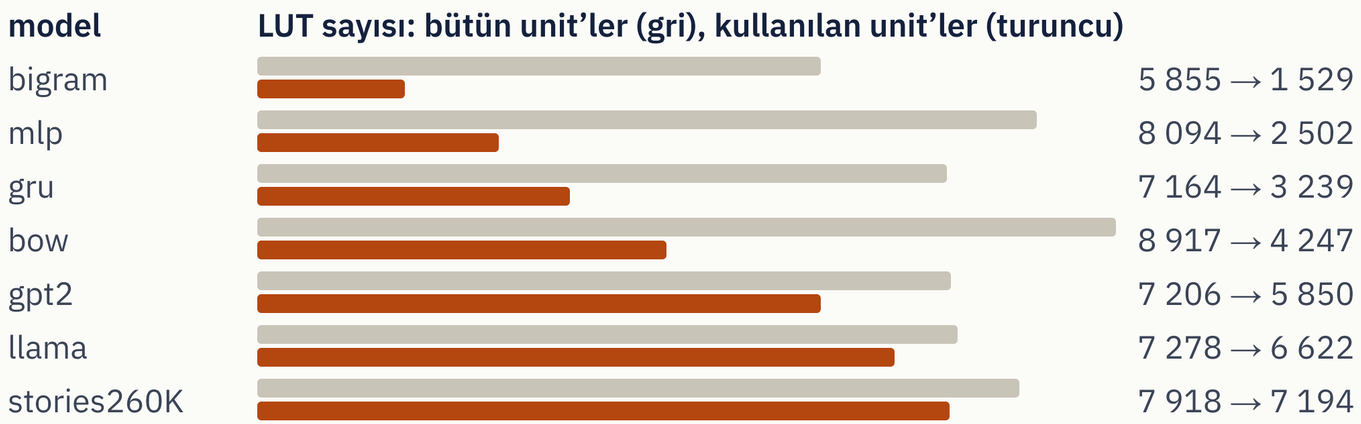
makemore modellerinde cycle'ların neredeyse tamamı GEMV'ye gider; transformer'larda norm ve attention da önemli pay alır



Şekil. Op türlerinin token başına cycle içindeki payı (Verilator profili); sağda engine'in token başına ortalama cycle sayısı. stories260K için T = 512 ve bütün pozisyonların ortalaması alınmıştır.

5 · ENGINE

SPECIALIZE ile modelin kullanmadığı unit'ler synthesis'te kaldırılır; makemore modellerinde engine dörtte birine küçülür



Mekanizma (zoo_core.sv)

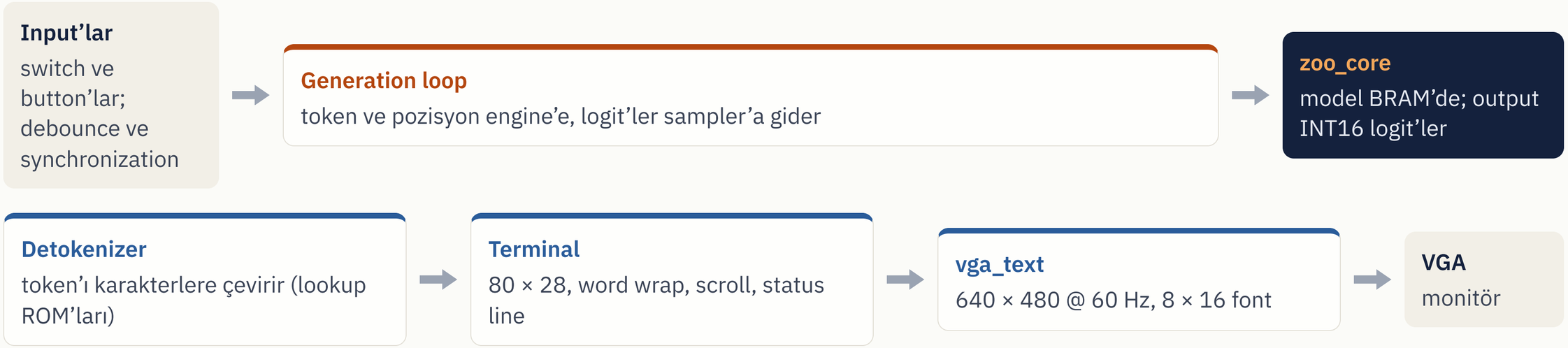
```
EN = SPECIALIZE ? OPS_USED
: 32'hFFFF_FFFF;
OP_NORM: if (EN[6]) ...
```

Compiler her model için hangi op'ların kullanıldığını bir bit maskesine yazar. Maskede sıfır olan unit'in logic'i sabit olarak kapalıdır ve synthesis aracı onu tamamen siler.

Değerler yalnızca engine için Yosys tahminidir. Bütün unit'leri içeren tek bitstream'lik sürüm 9 444 LUT kullanır.

6 · KART SİSTEMİ

Engine’in çevresindeki hardware, normalde bir bilgisayarın yaptığı generation loop, sampling, metin ve ekran işlerini üstlenir



On-board check

Token’ların hash’i, Python’un hesaplayıp bitstream’e gömdüğü değerle karşılaştırılır.

Overlap

Ekran yazımı bir sonraki token’ın hesabıyla paralel yürür; yalnızca bigram ekranı bekler.

7-segment, LED

7-segment gösterge token/s ya da cycle/token değerini, LED’ler token sayısını ve kontrol sonucunu gösterir.

6 · KART SİSTEMİ

Generation loop, engine’i çalıştıran, logit’leri üç kez tarayan ve bir sonraki token’ı seçen bir state machine’dir

G_STEP → **G_WAIT** → G_MAX → G_SUM → G_SEL → G_PICK → G_STEP …

durum	iş	cycle
G_STEP, G_WAIT	engine’e (token, pozisyon) ver, END’i bekle	engine’in cycle’ı
G_MAX	en büyük logit m; greedy ise argmax ile doğrudan G_PICK	V
G_SUM	$e_k = 2^{(-(m - l_k)c)}$, min-p kesmesi, Σe	V
G_SEL	thr = (r · Σe) ≫ 32; birikimli toplam thr’yi aşınca seç	≤ V
G_PICK	token’ı ekrana gönder, hash’i güncelle, pozisyonu artır	1

> **Sampler’ın ek maliyeti yaklaşık 3V cycle’dır.** stories260K’da bu 3 × 512 ≈ 1 500 cycle, yani token başına 50 397 cycle’ın %3’üdür; names modellerinde yaklaşık 80 cycle’dır.

Integer sampling, temperature ve min-p'yi softmax ile aynı LUT ile ve division kullanmadan uygular

$e_k = 2\text{^}(-(m - l_k) \cdot c)$, $c = s_\text{logit} \cdot \log_2 e / \tau$

kes: $e_k < 2^{15} \cdot 2\text{^}(-\text{NCUT})$ (min-p)

thr = $(r \cdot \Sigma\, e_k) \gg 32$, r: xorshift32

Division gerekmez. Probability'ler normalize edilmez; bunun yerine threshold, toplamla scale edilir. Random number 32 bitlik bir fraction gibi davranır: thr / Σe = r / 2^{32}.

τ switch'lerle 8 değerden, min-p 4 değerden seçilir; c her τ için fixed point olarak saklanır.

Örnek: 4 token, s_logit = 0,05, τ = 0,8, min-p = 1/32

l_k	u	e_k	integer p	float p (min-p)
120	0	32 768	0,7775	0,7773
100	462	9 379	0,2225	0,2227
60	1 385	770, kesilir	0	0
20	2 308	63, kesilir	0	0

Σe = 42 147'dir. r = 0,75 · 2^{32} için thr = 31 610 olur ve ilk token seçilir (32 768 > thr); r = 0,875 · 2^{32} için thr = 36 878 olur ve ikinci token seçilir.

Token’lar karakterlere çevrilir, kelimeler bölünmeden yazılır ve metin VGA output’unda her frame’de yeniden çizilir

Detokenizer

İki ROM kullanılır: birincisi her token’ın karakter dizisinin başlangıcını ve uzunluğunu, ikincisi karakterlerin kendisini tutar. stories260K’da bir token bir ile birkaç karakter arasındadır.

Terminal

Bir kelime 80. sütunu aşacaksa yarım kelime alt satıra taşınır. Ekran dolunca satırlar dairesel buffer’da kayar (scroll).

vga_text

Ekran 640 × 480 resolution’da, 60 Hz ve 25 MHz pixel clock’uyla çizilir; 80 × 30 karakter ve 8 × 16 pixel’lik font kullanılır. Üç stage’li pipeline metin RAM’ini, font ROM’unu ve pixel output’unu sırayla işler.

Status line

token/s değeri, f_clk / cycle_per_token olarak sequential bir divider ile hesaplanır; sayılar bin2bcd ile onluk sisteme çevrilir.



Şekil. VGA pinlerinden alınan frame; satır sonlarında kelimeler bölünmemiştir.

6 · KART SİSTEMİ

Switch’ler üretimi ayarlar, button’lar koşuları başlatır; LED16 kartın Python ile aynı sonucu ürettiğini gösterir

input	işlev
güç, CPU_RESET	ilk koşu kendiliğinden başlar
BTNC	yeni koşu; random durum sürer
BTNU	generator’ı SW[15:8]’den reseed et ve koş
BTND	metin alanını temizle
SW0	greedy ya da sampling
SW[3:1]	τ: 0,8 · 1,0 · 0,9 · 0,7 · 0,6 · 0,5 · 1,2 · 1,5
SW[5:4]	min-p: 1/32 · kapalı · 1/128 · 1/16
SW6, SW7	daktilo modu (~25 token/s); 7-segment: token/s ya da cycle/token

On-board check

`h ← 31 · h + tok`

Python, export sırasında greedy koşunun ve ilk sampling koşusunun hash’ini hesaplayıp bitstream’e gömer. Kart kendi hash’ini bununla karşılaştırır.

mavi: üretim sürüyor

yeşil: sonuç doğru

kırmızı: sonuç farklı

7 · LOW-BIT WEIGHT'LER

1-bit weight'ler memory'si sekiz kat küçültür ve çarpma işlemini toplama ile çıkarmaya dönüştürür

INT8

```
x = [12, −7, 30]
w = [18, −55, 127]
Σ wx = 4 411 3 çarpma
```

binary: w ≈ a · sign(w)

```
sign = [+, −, +]
12 + 7 + 30 = 49
y = a · 49 yalnızca 1 scale çarpması
```

Hocamızın önerdiği iki kaynak

LittleBit (Lee vd., NeurIPS 2025), weight matrix'ini low-rank binary factor'larla temsil ederek weight başına 1 bitin altına iner.
Larq, binary sinir ağları için bir kütüphanedir; quantizer'ları Bi-Real Net ve TWN yöntemlerini içerir.

Training ile inference'ın ayrılması

FPGA'de training yapılmaz. Low-bit weight'ler Mac'te QAT ile öğrenilir ve FPGA yalnızca sonucu çalıştırır. Literatürdeki uygulamaya uygun olarak embedding ve output head INT8 kalır.

7 · LOW-BIT WEIGHT'LER

Beş yöntem aynı training düzeniyle karşılaştırıldı; Larq'ın quantizer'ları ve LittleBit PyTorch'ta yeniden yazıldı

yöntem	weight	kaynağı	training ayrıntısı
int8	8 bit, per-channel scale	engine'in bugünkü formatı	training'siz (PTQ)
int4	4 bit, per-channel scale	standart QAT	STE ile rounding
tern	{−a, 0, +a}	Larq SteTern (TWN)	threshold 0,7 · ort|w|
bin	{−a, +a}, a = ort|w|	Larq MagnitudeAwareSign, SteSign	STE |w| ≤ 1, weight clip
LittleBit	diag(h) sign(U) diag(ℓ) sign(V)ᵀ diag(g)	Lee vd. 2025, Dual-SVID başlangıcı	rank %r, 1 ya da 2 path

STE (straight-through estimator)

İleri yönde w → sign(w); geri yönde gradient |w| ≤ 1 ise olduğu gibi geçer. Training gerçek değerli bir "gölge" weight'i günceller.

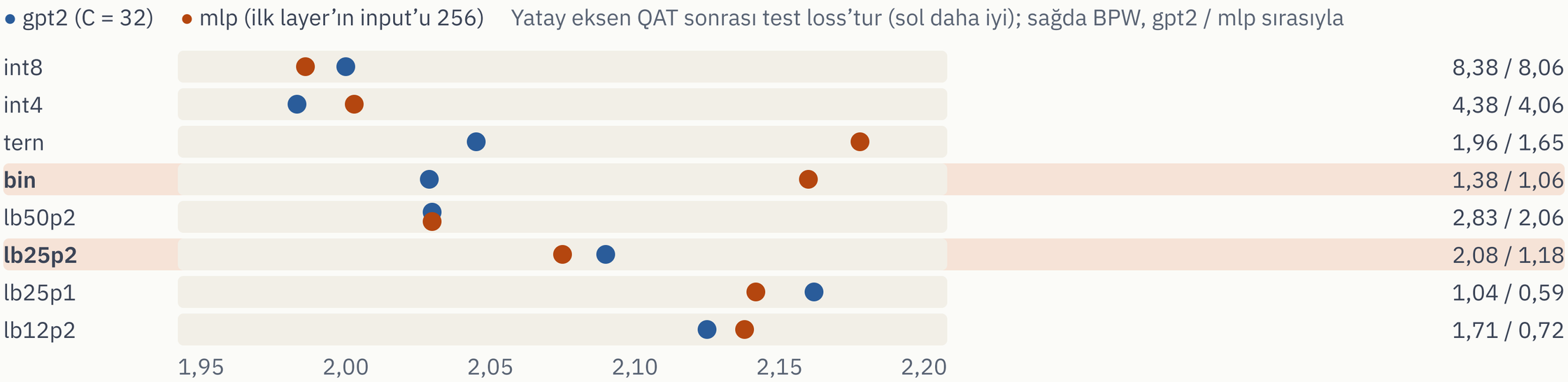
Knowledge distillation

Float model teacher olarak kullanılır; low-bit student model, sonraki token distribution'ına KL divergence ile yaklaştırılır. Optimizer olarak Adam ve cosine öğrenme oranı kullanılır.

PyTorch tercihi

Larq'ın quantizer'ları birkaç satırlık fonksiyonlardır; PyTorch'ta yeniden yazıldıklarında modeller ve integer akışı değişmeden kaldı.

Transformer'da binary, geniş input'lu MLP'de LittleBit daha iyi sonuç verir; INT4 her modelde float ile aynı kalitededir



Değerlendirme. gpt2'de binary 1,38 BPW ile 2,029 loss'a ulaşır (float 2,000) ve LittleBit aynı bütçede geride kalır. mlp'de ise LittleBit 1,18 BPW ile 2,075'e, binary 1,06 BPW ile 2,160'a ulaşır.

Bunun nedeni, LittleBit'in scale vector'lerinin dar layer'larda bit bütçesinin büyük kısmını kullanmasıdır

$$\text{BPW} = k \cdot [\, r(d_o + d_i) + 16(d_o + d_i + r) \,] \,/\, (d_o\, d_i)$$

$$\text{binary: } 1 + 16 \,/\, d_i$$

İlk terim sign bitlerini, ikinci terim 16 bitlik h, g ve ℓ scale vector'lerini sayar; *k* path sayısı, *r* ise rank'tır. Scale'lerin payı layer genişliği *d* ile 1/*d* oranında azalır.

> Makaledeki 0,1 BPW sonucu 4096 × 4096 boyutlu, r = 86 ve iki path'li bir layer'a aittir (0,0998 BPW); bu genişlikte scale payı ihmal edilebilir.

layer	binary	yalnız scale	r %25, 1 path	r %25, 2 path
32 × 32 (names)	1,50	1,00	1,63	3,25
64 × 64 (stories)	1,25	0,50	1,06	2,13
172 × 64	1,25	0,34	0,71	1,42
4096 × 4096	1,00	0,008	0,51	1,02

Değerlendirme. 32 genişliğindeki bir layer'da yalnızca scale'ler 1,00 BPW tutar; bu, binary bir layer'ın toplam bedelinin üçte ikisidir. LittleBit'in bit bütçesi rank yerine scale'lere gider; binary ise aynı bütçeyle tam rank'lı bir sign matrix'i kullanır.

7 · LOW-BIT WEIGHT'LER

Engine'e 64 lane'li ve multiplier'sız bir 1-bit GEMV unit eklendi; binary ve LittleBit modelleri aynı unit'te çalışır

GEMVB (opcode 18)

wbin.mem'deki 64 bitlik bir word, bir input'un 64 output için sign'larını taşır (1 = +1, 0 = −1).

64 toplama-çıkarma lane'i bulunur ve DSP kullanılmaz. Accumulator'lar 20 bittir, çünkü $|acc| \le 128 \cdot n_in < 2^{19}$.

Seri requant ve control logic INT8 GEMV ile ortaktır. Channel scale'i a, requant multiplier'ı $m_0 \cdot 2^{-sh}$ içine fold edilir.

SCALE (opcode 17)

Norm'un γ'sı normalde weight'e fold edilir ($W \cdot diag(\gamma)$); bu işlem binary weight'i bozar.

$$W \cdot diag(\gamma) = diag(a) \cdot S \cdot diag(g),\ S \in \{\pm 1\}$$

diag(a) requant'a aktarılır; g ise q, k ve v için ortak tek bir SCALE op'una dönüşür: $y_i = requant(x_i \cdot g_i)$.

LittleBit tek path

Aynı unit ile üç op yeterlidir:

```
SCALE g·γ
GEMVB sign(V)ᵀ, ℓ
GEMVB sign(U), h
```

ℓ ve h ilgili requant multiplier'larına fold edilir.

Unit yalnızca modelde GEMVB op'u varsa synthesize edilir (SPECIALIZE). İlk sürüm 14 709 LUT kullanıyordu; accumulator'lar ayrı bir 20 bitlik bankaya alındığında bu değer 11 153'e indi (gpt2-bin, tüm kart).

7 · LOW-BIT WEIGHT'LER

Kartta low-bit modeller 1,7–3,6 kat hızlıdır ve daha az BRAM kullanır; bunun karşılığında loss artar

model	cycle / token	hız	BRAM36	LUT	test loss
gpt2 (INT8)	8 713		19,5	7 112	2,002
gpt2-bin	4 983	1,7×	10	11 153	2,028
mlp (INT8)	6 722		14	4 949	1,987
mlp-lb25p1	1 874	3,6×	5	7 940	2,142
stories260K (INT8)	50 397		97,5	9 002	1,291
stories260K-bin	27 992	1,8×	56	12 118	1,927

Hız

64 lane, 8 lane'den sekiz kat geniştir; norm ve attention gibi diğer op'lar değişmediği için toplam kazanç 1,7–3,6 kattır.

Memory ve LUT

BRAM kullanımı %43–64 azalır; toplama lane'leri LUT'larla kurulduğu için LUT kullanımı %35–60 artar.

Kalite

Loss, names'te binary ile 0,03, LittleBit ile 0,16 nat artar. stories260K'da artış 0,64 nattır; 260 bin parametre bu görev için zaten sınırdadır.

8 · VERIFICATION

Verification sekiz aşamalı bir zincir olarak kuruldu; her aşama bir öncekinin çıktısını referans alır

1 · Float ↔ Karpathy
fark < 10^{-4}

2 · Integer ↔ float
loss farkı ≤ 0,022

3 · RTL ↔ integer
tam kart, 18/18 bit-exact

4 · Random test'ler
655 koşu, ~121 bin token

5 · 4-state
Icarus; bilinmeyen değer yok

6 · Netlist ↔ RTL
kapı seviyesi; tool bug'ları

7 · Timing
nextpnr; statik analiz

8 · Kartta
kapsam dışı

Bit-exact ölçütü

Hardware ve referans aynı integer işlemlerini yapar. Yaklaşık bir karşılaştırma bug'ları gizleyebilir; bit-exact karşılaştırmada tek bir farklı token bug'ı ortaya çıkarır.

Fiziksel kart olmadan yeterlilik

Tasarım tek clock'lu ve synchronous'tur. Netlist functionally doğruysa ve her path bir clock period'una sığıyorsa, chip'teki circuit simülasyondaki gibi davranır.

8 · VERIFICATION

Tam kart simülasyonunda testbench bir kullanıcı gibi button’lara basar ve yalnızca pinlerden okunabilen değerleri karşılaştırır

karşılaştırılan	nasıl
her token	GENTOK satırları ↔ board.py:generate
hash, LED16	on-board check’in sonucu
ekran metni	80 × 30 karakter, her boşta kalışta
VGA frame’i	R, G, B ve sync pin’lerinden
7-segment, LED	token sayacı, hız göstergesi
pin imzası	her cycle’da bütün pinler

Her model için dört koşu yapılır: otomatik sampling koşusu, BTNC ile ikinci koşu, SW0 açıkken BTNU ile greedy koşu ve BTND ile ekran temizleme. Input’lar clock’un falling edge’inde drive edilir, output’lar yine falling edge’de sample edilir.

stories260K için testbench çıktısı

```
run 0: 256 tokens (identical)
hash 44fe319f = 44fe319f
screen, status, check, LEDs,
7-segment OK
VGA frame: 0 sync errors,
0 pixels differ
```

bigram	mlp	rnn
gru	bow	gpt2
microgpt	llama	nanochat
stories260k	ts_gpt2	ts_gru
ts_llama	ts_microgpt	ts_nanochat
gpt2-bin	mlp-lb25p1	stories-bin

18 modelin 18’i bütün karşılaştırmaları geçti; toplam 11 241 token.

8 · VERIFICATION

Random test’ler ve corner case testleri, normal kullanımda ortaya çıkmayan iki RTL bug’ını tespit etti

655

random koşu

Yaklaşık 121 bin token üretildi; koşu sırasında button ve CPU_RESET kullanıldı; 32 sampling ayarının 30–32’si kapsandı.

16

corner case

Koşu sırasında basılan button’un queue’ya alınması ve reset sonrası token’ların referansla örtüşmesi sınandı.

0 X

4-state simülasyon

Icarus ile gru modelinde 2,7 milyon cycle simüle edildi; pinlerde ve metin RAM’inde bilinmeyen değer görülmedi.

Bug 1: kaybolan button

Queue’daki bir koşunun başladığı cycle’a denk gelen BTNU basışı kayboluyordu. Queue logic’i düzeltildi.

Bug 2: reset’i olmayan counter

Heartbeat sayacının reset’i yoktu ve CPU_RESET onu sıfırlamıyordu. Bug, 2-state Verilator’da görünmüyordu; 4-state simülasyon LED0’da bilinmeyen değer gösterdi.

8 · VERIFICATION

Netlist simülasyonu, RTL doğru olsa bile synthesis araçlarının hata yapabileceğini gösterdi; üç tool bug'ı tespit edildi

sv2v 0.0.13

```
64'(att_a[j]) >>> 32
```

Signed bir array element'ine uygulanan boyut cast'ini unsigned çeviriyordu; arithmetic shift logical shift'e dönüşüyor ve negatif değerler +127'ye doyuyordu.

Çözüm: ifade açık $signed(...) ile yazıldı.

Yosys 0.69: wreduce + peepopt

```
va_d[8*(((h<<3)+d) % P) +: 8]
```

İki optimizasyon adımı art arda çalıştığında q·k toplamındaki dynamic byte select'i bozuluyor ve t = 0'dan sonraki attention output'ları yanlış hesaplanıyordu.

Çözüm: önce head dilimi seçildi, ardından sabit indeksler kullanıldı.

Yosys 0.69: DSP48E1

```
prod_r <= sum * m0_s;
```

Input'ları register'dan gelen requant multiply'ını DSP'ye map ederken sign extension'ını düşürüyor; negatif accumulator'lar en büyük değere doyuyor.

Çözüm: netlist ve open-source bitstream'de multiplier'lar LUT'larla kuruldu (-nodsp).

Sonuç. Gerçek üst modülün Yosys netlist'i (MMCM, BUFG ve BRAM dahil) gru, gpt2-bin ve mlp-lb25p1'de her pinde ve her cycle'da RTL ile aynıdır; gpt2-bin'de 4 200 047 cycle boyunca VGA frame'i de aynı kaldı. Verilator ve Vivado SystemVerilog'u doğrudan okuduğu için ilk iki bug'dan etkilenmez.

9 · PLACE AND ROUTE VE HIZ

Synthesis, placement, routing, timing analysis ve bitstream üretimi Vivado olmadan, Mac'te open-source araçlarla yapıldı

Yosys
LUT, FF ve BRAM netlist'i

→

nextpnr
placement, routing, Fmax

→

FASM
yapılandırma listesi

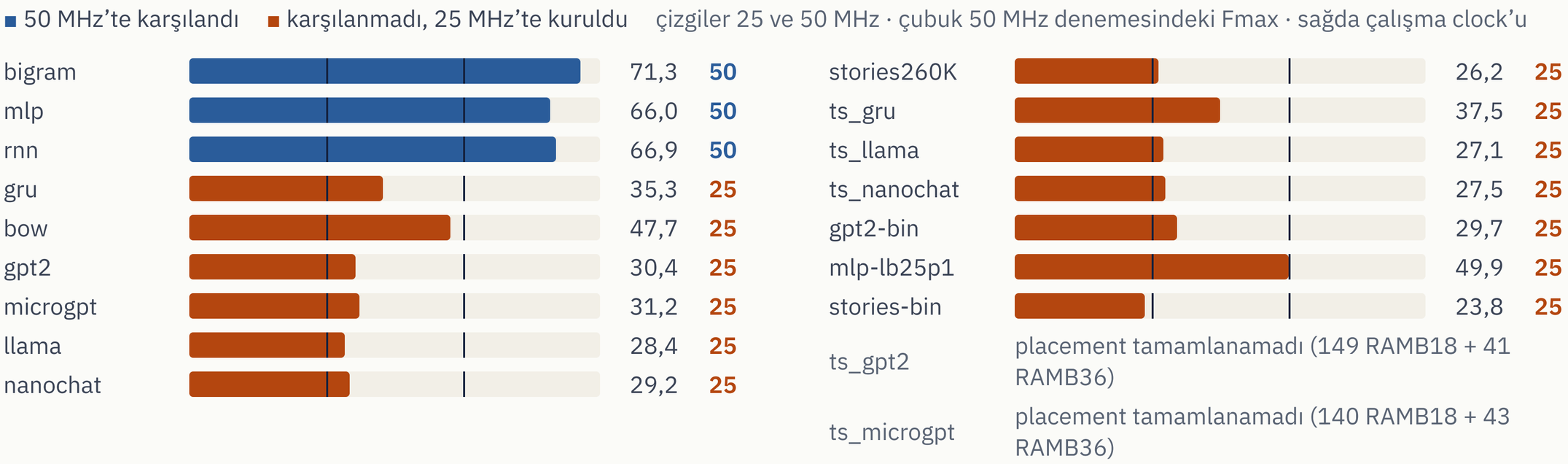
→

Project X-Ray
frame'ler ve .bit dosyası

sorun	çözüm
macOS SDK'sı Xcode linker'ıyla okunamıyor	derlemede Xcode'un SDK'sı kullanıldı (SDKROOT)
nextpnr pin adlarını boş okuyordu	PACKAGE_PIN boşalmış bir referanstan kopyalanıyordu; XDC'de doğrudan LOC yazıldı
50 MHz tutmadığında akış duruyordu	timing raporu her durumda yazılıyor; tutmazsa tasarım 25 MHz'te yeniden kuruluyor
DSP mapping'inde Yosys bug'ı	multiplier'lar LUT'larla kuruldu (-nodsp); bu netlist simülasyonla bit-exact olarak doğrulandı

Akış bigram için uçtan uca 1 dakika sürer (3,8 MB bitstream); stories260K için 50 MHz denemesi ve 25 MHz'te yeniden kurulumla birlikte yaklaşık 13 dakikadır.

makemore’un üç modeli 50 MHz’te timing’i karşılar; transformer’lar 26–31 MHz’te kalır ve 25 MHz’te çalışır

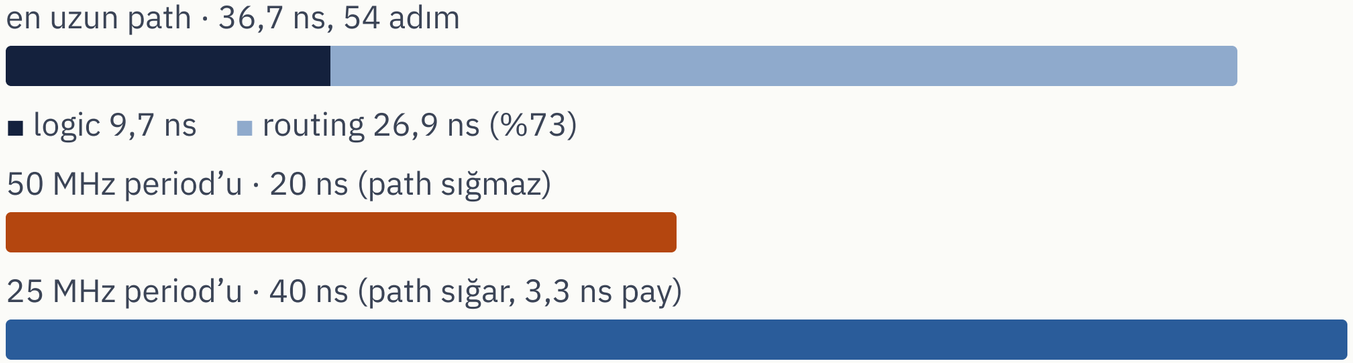


Bulgu. Yerleşen 16 modelin tamamı çalıştığı clock’ta timing constraint’lerini karşılar. Fmax placement seed’ine göre değişir: bow’un 25 MHz kurulumu bir seed ile 50,1, diğeriyle 44,0 MHz verdi.

Critical path, buffer'dan okuma, çarpma ve geri yazmanın tek cycle'da yapılmasından kaynaklanır ve 37 ns sürer

j, f_n0 → va_a: buffer address'i → LUTRAM okuma, qh → LUT multiplier, carry → vw_d: yazma

Şekil. stories260K'nın critical path'i ve clock period'ları (aynı scale'de)



Path modele göre değişir, yapısı aynıdır

stories260K, llama ve nanochat'te path attention'ın head diliminden (qh), gpt2-bin'de NORM output'undaki d · inv multiply'ından ve değişken shift'ten geçer.

Çözüm önerisi

Okuma ile hesap arasına bir register eklenmesi path'i ikiye böler ve op'ları bir cycle geciktirir. Hedef 50 MHz'tir.

Place and route sonrasında en büyük model bile chip’in logic resource’larının üçte birinden azını kullanır

model	LUT bel	FF	CARRY4	BRAM36	DSP
bigram	6 362 (%5)	1 334	242	2 (%1)	0
mlp	13 631 (%11)	2 228	415	14 (%10)	0
gru	15 506 (%12)	2 255	479	16 (%12)	0
gpt2	29 435 (%23)	3 265	773	19,5 (%14)	0
nanochat	32 288 (%25)	3 416	840	16 (%12)	0
stories260K	34 643 (%27)	3 513	862	97,5 (%72)	0
ts_gru	15 826 (%12)	2 289	475	69,5 (%51)	0
gpt2-bin	35 041 (%28)	5 958	1 106	10 (%7)	0
stories260K-bin	39 334 (%31)	6 120	1 198	56 (%41)	0

Tablo. LUT bel, chip’teki 126 800 LUT yuvasından kullanılanı gösterir; BRAM36 = RAMB36 + RAMB18/2 ve Yosys tahminiyle aynıdır. Multiplier’lar LUT’larla kurulduğu için DSP kullanımı sıfırdır; Yosys’in DSP’li tahmini stories260K için 9 002 LUT ve 66 DSP’dir.

Gerçek hız, clock frequency'sinin token başına cycle sayısına bölümüdür; stories260K saniyede 496 token üretir

model	MHz	cycle/token	token/s
bigram	50	157	318 066
mlp	50	6 722	7 438
rnn	50	6 464	7 735
gru	25	6 605	3 785
bow	25	6 368	3 926
gpt2	25	8 713	2 869
microgpt	25	8 746	2 858
llama	25	9 491	2 634

model	MHz	cycle/token	token/s
nanochat	25	10 813	2 312
stories260K	25	50 397	496
ts_gru	25	30 407	822
ts_llama	25	50 397	496
ts_nanochat	25	50 680	493
gpt2-bin	25	4 983	5 017
mlp-lb25p1	25	1 874	13 340
stories-bin	25	27 992	893

Karşılaştırma ölçeği. Bir insan saniyede birkaç token okuyabilir; kart en yavaş modelde bile bu hızın yaklaşık yüz katıyla yazar. Bu nedenle kartta saniyede ~25 token'lık bir daktilo modu da bulunur. Pipeline ile 50 MHz'e çıkılırsa 25 MHz'teki bütün değerler iki katına çıkar.

10 · SONUÇ

Kalite kaybının tek kaynağı INT8 quantization'dır; FPGA bu kayba hiçbir şey eklemez



Şekil. stories260K için loss'un aşamalara göre değişimi. Names modellerinde quantization'ın bedeli en fazla 0,0055 nat'tır (s. 17).

Hız: aynı model, CPU ve FPGA

platform	clock	token/s	cycle/token
M1 CPU · run.c, float32, 1 thread	3,2 GHz	14 397	222 273
FPGA · INT8 engine	25 MHz	496	50 397
FPGA · pipeline hedefi	50 MHz	~990	50 397

Mutlak hızda dizüstü CPU 29 kat önde, çünkü clock'u 128 kat yüksek. Token başına harcanan cycle'da ise FPGA 4,4 kat daha verimli.

Henüz iddia etmediklerimiz

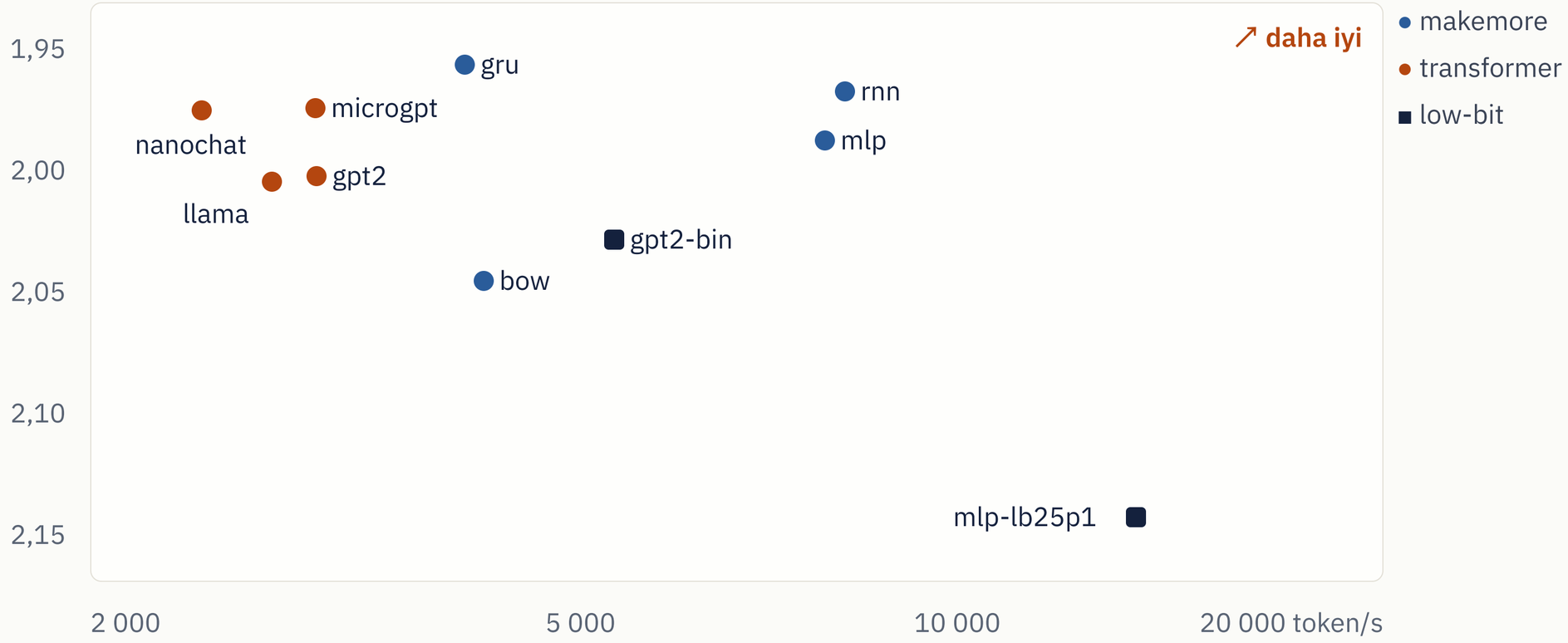
Enerji ölçülmedi; kart ve Vivado güç raporu olmadan token/J verilemez.

Fmax, Project X-Ray'in timing modeline dayanır; Vivado ile doğrulanmadı.

Multiplier'lar DSP yerine LUT'larda kuruldu; hızdaki en büyük pay pipeline'da.

10 · SONUÇ

Kalite ve hız birlikte değerlendirildiğinde names’te en iyi loss gru’da, en iyi denge rnn ve mlp’dedir



47 · Karpathy modelleri FPGA’de · dikey: test loss, yukarı iyi · yatay: token/s (log) · bigram dışarıda

10 · SONUÇ

Kullanılan tekniklerin çoğu literatürde bilinmektedir; çalışmanın özgünlüğü birleşimde, ölçüm düzeneğinde ve verification'dadır

Bizim katkımız

1. **Eşit bütçe karşılaştırması.** Dokuz mimarinin her tasarım fikrinin (RMSNorm, RoPE, GQA, SwiGLU, QK-norm) cycle, resource, Fmax ve kalite üzerindeki etkisi ayrı ayrı ölçüldü.
2. **Tek microcoded engine.** RNN'den nanochat'e on model, Karpathy'nin stories260K'sı dahil, compiler'ıyla birlikte tek bir engine'de bit-exact çalışıyor.
3. **Bağımsız kart.** Sampling dahil bütün işlemler integer ile FPGA'de yapılıyor; host bulunmuyor ve kart sonucunu kendisi kontrol ediyor.
4. **İspatlı sınırlar ve sıkı verification.** Bütün işlemlerin range'leri ispatlandı, hardware pin seviyesinde bit-exact doğrulandı ve dört tool bug'ı bulundu.
5. **LittleBit'in küçük ölçekteki sınırı.** Scale maliyetinin etkisi formülle ve ölçümle gösterildi; binary ve LittleBit aynı 1-bit unit'te çalışıyor.

Literatürden aldıklarımız

requant, per-channel scale (Jacob vd., 2018)

integer softmax ve LayerNorm (I-BERT, Softermax)

output-stationary GEMV (Sze vd.)

komutla programlanan engine (VTA, NPE)

STE, binary, ternary (Larq, Bi-Real Net, TWN)

LittleBit (Lee vd., 2025); percentile kalibrasyon

En yakın çalışmalar. HLSTransform llama2.c'nin 110M modelini çalıştırır, ancak weight'ler external memory'de, sampling host'tadır. TALOS-V2 tek bir microGPT modelini JTAG üzerinden sürer. COBRA'nın 1-bit unit metin üretmez. Değerlendirme Ekim 2026 literatür taramasına dayanır.

10 · SONUÇ

Çalışmanın sınırları: kanıtlanan ve henüz kanıtlanmayan noktalar

Fiziksel kart testi yapılmadı

Kanıt, simülasyona ve timing analysis'e dayanır. Place and route sonrasındaki son circuit simüle edilemedi; nextpnr bu circuit'i Verilog olarak yazamıyor.

Fmax değerleri yaklaşıktır

nextpnr'ın delay modeli topluluk tarafından çıkarılmıştır ve Vivado'nun sonucu farklı olabilir. Multiplier'lar LUT'larla kurulduğu için circuit, Vivado'nun DSP'li circuit'inden daha yavaş ve daha büyüktür.

Kapasite sınırı

607 KB BRAM yaklaşık 300 bin parametreye kadar yeterlidir. nanoGPT ve llm.c'nin hazır checkpoint'leri (10 milyon ve üzeri) için DDR2'den weight okuyan farklı bir mimari gerekir.

Tamamlanmayan parçalar

ts_gpt2 ve ts_microgpt chip'e place edilemedi. gpt-nano sıralama demosu bu kart tasarımında bulunmuyor. Larq paketiyle sayısal karşılaştırma ve iki path'li LittleBit hardware'i henüz yapılmadı.

10 · SONUÇ

Sonraki adımlar öncelik sırasıyla hız, verification'ın tamamlanması ve kapsamın genişletilmesidir

1. **Engine'e pipeline eklenmesi:** buffer okuma ile hesap arasına bir register konacak. Hedef 50 MHz, yani token/s'de iki kat; 18 modelin simülasyonları yeniden geçirilecek.

2. **Place edilen netlist'in simülasyonu:** 18 modelin tamamında, gerçek clock ayarı ve gerçek debounce süreleriyle yapılacak.

3. **DSP'ler ve memory layout:** Yosys bug'ı geliştiricilere raporlanacak, requant dışındaki multiplier'lar DSP'ye taşınacak; ts_gpt2 ve ts_microgpt'nin memory'leri RAMB36'ya uygun biçimde bölünecek.

4. **19. model:** minGPT'nin gpt-nano sıralama demosu eklenecek; ekranda girdi dizisi ve sıralanmış çıktı gösterilecek.

5. **Low-bit weight'ler:** stories260K için daha uzun bir binary QAT yapılacak ve iki path'li LittleBit hardware'de denenecek.

ÖZET

1 · Karpathy’nin dokuz mimarisi ve stories260K, tek bir microcoded integer engine’inde çalışıyor; modelden modele yalnızca BRAM’deki microcode ve weight’ler değişiyor.

2 · Kart bilgisayarsız çalışıyor: generation loop, integer sampling, metin ve VGA ekranı FPGA’de; 18 modelin tamamı Python referansıyla bit-exact.

3 · Binary weight’ler 1,7–3,6 kat hızlanma ve daha az memory sağladı; LittleBit küçük modellerde yalnızca geniş layer’larda binary’den üstün.

4 · Open-source akışla 16 model chip’e place edildi; makemore’un üç modeli 50 MHz’te, diğerleri 25 MHz’te çalışıyor. Engine’e pipeline eklenerek hedef 50 MHz.

Teşekkürler. Sorular?

github.com/Muratatceken/mingpt-fpga